

Plano Consolidado de Melhoria Estrutural: Arquitetura de Diretórios e Governança Agêntica

Fusão estratégica entre o plano original, auditorias técnicas OMP GLM 5.2 e Claude Sonnet 5

Fábrica Agêntica: Antigravity + Gemini 3.6 Flash

2026-08-05

Plano Consolidado de Melhoria Estrutural: Arquitetura de Diretórios e Governança Agêntica

Este documento é a fusão técnica e definitiva do plano original (docs/04-plano-melhoria-estrutural.md) com as revisões críticas emitidas pelas auditorias OMP (nvidia/z-ai/glm-5.2) e Claude Sonnet 5. O objetivo é estabelecer uma arquitetura de governança enxuta, eliminar a duplicidade física de skills, resolver a dívida de portabilidade e garantir um pipeline 100% determinístico.

Diagnóstico Factual Consolidado e Mapeamento de Riscos

Após auditoria detalhada no repositório em 2026-08-05, foram constatados os seguintes achados e pontos de atenção:

1. Duplicidade Massiva em Arquivos de Orquestração

- **Diagnóstico Real:** A duplicidade principal ocorre entre CLAUDE.md (219 linhas) e AGENTS.md (174 linhas), que compartilham 93 linhas idênticas (~95% de sobreposição funcional).
- **Esclarecimento Factual:** GEMINI.md possui 46 linhas e já é ultra-específico (focado em Caveman Thinking e hooks do code-review-graph), não contendo a duplicidade citada no diagnóstico original.
- **Omissão Corrigida:** O projeto possui outros quatro arquivos de governança de 38 linhas cada (CODEBUDDY.md, QODER.md, .cursorrules, .windsurfrules) que precisavam ser integrados ao modelo centralizado.

2. Duplicidade Física de Skills e Fragilidade no Windows

- **Situação Atual:** As 4 skills comuns (debug-issue, explore-codebase, refactor-safely, review-changes) estão copiadas fisicamente nas pastas .claude/skills/, .gemini/skills/ e .codebuddy/skills/.
- **Risco dos Symlinks Nativos:** A utilização de os.symlink no Windows requer privilégios de Administrador ou Modo Desenvolvedor ativo, o que causaria falhas silenciosas em ambientes CI/CD ou máquinas de operadores.

- **Solução Definida:** Utilização de **Directory Junctions** (`mklink /J`) no Windows como mecanismo primário (não exige elevação de privilégio), acompanhado de script de setup idempotente que **nunca realiza exclusões destrutivas** antes de confirmar a integridade da vinculação.

3. Portabilidade Quebrada por Caminhos Absolutos

- **Achado Crítico:** Foi identificada a presença de caminhos absolutos hardcoded com referências à máquina local (`C:\Users\trcnologia\...`) em 9 arquivos do ecossistema:
 - `.gemini/hooks/crg-session-start.sh`
 - `.gemini/hooks/crg-update.sh`
 - `.gemini/settings.json`
 - `.mcp.json` (raiz)
 - `.cursor/mcp.json`
 - `.vscode/mcp.json`
 - `.kiro/settings/mcp.json`
 - `.qoder/mcp.json`
 - `opencode.jsonc`
-

Proposta Arquitetural e Divisão de Responsabilidades

Tabela de Especialização dos Arquivos de Governança

Arquivo	Papel Estratégico	Conteúdo Exclusivo	Meta de Tamanho
AGENTS.md	Bíblia Conceitual da Fábrica (Única fonte de verdade)	Visão geral da fábrica agêntica, regras invariáveis de negócio, fluxo do pipeline, tabela completa de módulos (incluindo “Textos de Apoio” e colunas do / esboçar), pré-requisitos de ambiente e catálogo de skills.	~180 linhas
CLAUDE.md	Orquestração Estrita do Claude	Instruções de comandos do Claude CLI (/ esboçar, /produzir-comunicacao-completa), ferramentas nativas do Claude e link de referência para AGENTS.md.	~35 linhas
GEMINI.md	Orquestração Estrita do Gemini	Padrões de economia de tokens do Gemini (Caveman Thinking), pre-flight rules, hooks de sessão (crg-session-start.sh) e integração com code-review-graph.	~45 linhas
CODEBUDDY.md	Especialização CodeBuddy	Diretivas específicas para a IA do CodeBuddy e ponte para AGENTS.md.	~25 linhas
QODER.md	Especialização Qoder	Diretivas específicas para o Qoder e ponte para AGENTS.md.	~25 linhas
.cursorrules	Especialização Cursor	Regras de contexto para a IDE Cursor apontando para AGENTS.md.	~25 linhas
.windsurfrules	Especialização Windsurf	Regras de contexto para a IDE Windsurf apontando para AGENTS.md.	~25 linhas

Plano de Implementação Consolidado Passo a Passo

O plano será executado sequencialmente do início ao fim, utilizando os scripts determinísticos de auditoria do repositório como gates de aprovação.

Etapa 1: Governança, Limpeza de Tokens & Portabilidade de Caminhos

- ↳ Preservar tabela completa em AGENTS.md + Enxugar CLAUDE.md/outros + Remover caminhos hardcoded
- ↳ Gate: `python scripts/auditar-projeto.py --estrito`

Etapa 2: Centralização de Skills & Script de Setup Idempotente (Junctions)

- ↳ Criar /skills na raiz + Criar scripts/setup-workspace.py + Validar hooks crg
- ↳ Gate: `python scripts/verificar-consistencia-pipeline.py --estrito`

Etapa 3: Governança de Configurações MCP, Validação & Geração dos Artefatos

- ↳ Sanitizar arquivos MCP + Executar auditoria integral + Gerar saídas em .md e .pdf

Detalhamento das Etapas de Execução

Etapa 1: Harmonização de Governança e Portabilidade

1. **Consolidação em AGENTS.md:** Garantir que AGENTS.md contenha o núcleo de regras invioláveis, o grafo de conhecimento e a tabela de módulos preservando todas as colunas.
2. **Refatoração de CLAUDE.md e Outros:** Reduzir CLAUDE.md para um arquivo direto e enxuto de instrução rápida. Padronizar CODEBUDDY.md, QODER.md, .cursorrules e .windsurfrules.
3. **Higienização de Caminhos Absolutos:** Substituir referências C:\Users\trcnologia\... por referências relativas baseadas no workspace ou variáveis de ambiente nos 9 arquivos catalogados.
4. **Gate Determinístico 1:** Executar `python scripts/auditar-projeto.py --estrito`.

Etapa 2: Centralização de Skills via Directory Junctions

1. **Diretório Raiz skills/:** Estabelecer a pasta skills/ na raiz do repositório contendo as versões únicas e autoritativas de debug-issue, explore-codebase, refactor-safely e review-changes.
2. **Desenvolvimento do scripts/setup-workspace.py:**
 - Detectar SO (Windows vs Unix).
 - No Windows, criar Directory Junctions (`mklink /J`) de skills/ para .claude/skills, .gemini/skills, .codebuddy/skills, etc.
 - Operar de forma segura: validar que a fonte existe, criar o link e testar a resolução antes de qualquer remoção de cópias sobressalentes.
3. **Verificação do Grafo de Conhecimento:** Validar que os hooks .gemini/hooks/crg-update.sh e crg-session-start.sh operam perfeitamente sem erros de caminho.
4. **Gate Determinístico 2:** Executar `python scripts/verificar-consistencia-pipeline.py --estrito` e `python scripts/auditar-projeto.py --estrito`.

Etapa 3: Governança MCP e Finalização

1. **Harmonização dos Schemas MCP:** Manter as especificidades de schema por plataforma em `.mcp.json`, `.cursor/mcp.json`, `.vscode/mcp.json`, `.kiro/settings/mcp.json` e `.qoder/mcp.json` com caminhos limpos e dinâmicos.
 2. **Compilação do Documento Final:** Salvar as versões `.md` e `.pdf` oficiais em `docs/`.
 3. **Validação Final:** Garantir 100% de conformidade com os scripts de testes e auditoria do projeto.
-

Critérios de Sucesso e Validação Automática

- **Redução da Pegada de Preâmbulo:** Redução de ~80% no consumo de tokens de inicialização do Claude Code por sessão.
- **Portabilidade Total:** Zero ocorrências de caminhos absolutos locais no repositório.
- **Sincronia de Skills:** Alterações na pasta `skills/` da raiz refletem instantaneamente em todos os agentes conectados via Directory Junctions.
- **Aprovação sem Avisos:** Execução bem-sucedida de `auditar-projeto.py --estrito` e `verificar-consistencia-pipeline.py --estrito`.